

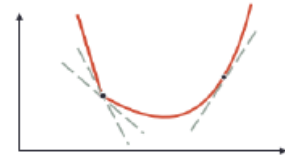
{HW4}

Part A:

Question 1

Definition: the set of **subgradients** of $f: V \rightarrow \mathbb{R}$ at point $u \in V$ is:

$$\partial f(u) \triangleq \{q \in V | \forall v \in V: f(v) \geq f(u) + q^T(v - u)\}.$$



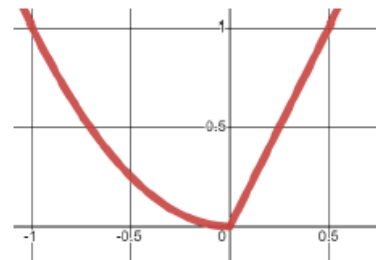
1. Let $f(x) = \begin{cases} x^2, & x < 0 \\ 2x, & x \geq 0 \end{cases}$.

1.1. Is f convex? No need to explain.

1.2. Propose a sub-derivative function g for f . That is, $g \in \partial f$.

Use the above definition to prove that $g(u) \in \partial f(u), \forall u \in \mathbb{R}$.

1.3. Set a learning rate of $\eta = 0.25$ and a starting point $x_0 = -1.5$.



Running subgradient descent, will the algorithm converge to a minimum?

Prove your answer by filling the following table like we did in Tutorial 07 using as many rows as needed.

i	x_i	$f(x_i)$	$\frac{\partial}{\partial x} f(x_i) = g(x_i)$
0	-1	1	
1			
$\vdots$			

1.4. Repeat 1.3 with $\eta = 1, x_0 = -1.5$.

1.1 Yes $f(x)$ is convex.

1.2 Define $g(x) = \begin{cases} 2x, & \text{if } x < 0 \\ 0, & \text{if } x = 0 \\ 2, & \text{if } x \geq 0 \end{cases}$

Said $g(x)$ is a sub-derivative function since it agrees with $f'(x)$ where it is differentiable. And where it is not we have $g(0) = 0$ which satisfies what is needed.

All we essentially needed here is some: $\alpha \in [0, 2]$

to plug in for $g(0) = \alpha$, since $f'(0^-) = 0$ and $f'(0^+) = 2$

1.3 $x_0 = -1.5, \eta = 0.25$

i	x_i	$f(x_i)$	$g(x_i)$
0	-1.5	2.25	-3
1	-0.75	$\frac{9}{16}$	-1.5
2	-0.375	$\frac{9}{64}$	-0.75
3	$-\frac{3}{16}$	$\frac{9}{256}$	-0.375

1.4 $x_0 = -1.5, \eta = 1$

i	x_i	$f(x_i)$	$g(x_i)$
0	-1.5	2.25	-3
1	1.5	2.25	2
2	-0.5	0.25	-1
3	0.5	0.25	2
4	-1.5	2.25	-3

We don't get convergence since $x_0 = x_4$, meaning we are stuck in a loop.

Part B:

Question 2

Data: $(x, y) \in \{(0,0), (1,3), (2,12)\}$

Hypothesis class: $\mathbb{H} = \{w_0 + w_1x + w_2x^2 + w_3x^3 \mid w_i \in \mathbb{R}\}$

2.1:

For $w_0 = 0, w_1 = 2, w_2 = 0, w_3 = 1$ we get the polynomial given by: $p(x) = x^3 + 2x$

We get:

$$p(0) = 0^3 + 2 * 0 = 0$$

$$p(1) = 1^3 + 2 * 1 = 3$$

$$p(2) = 2^3 + 2 * 2 = 8 + 4 = 12$$

2.2:

The polynomial $f(x) = 3x^2$ fits the data perfectly and is quadratic.

2.3:

For quadratic model, $w_{2.2} = (0,0,3,0)$

For the cubic model, $w_{2.1} = (0,2,0,1)$

Therefore: since $m = 3$

- $l(w_{2.2}) = \frac{1}{3} \left(\sum_{i=1}^3 (3x_i^2 - y_i)^2 \right) = 0$
- $l(w_{2.1}) = \frac{1}{3} \left(\sum_{i=1}^3 (x_i^3 + 2x_i - y_i)^2 \right) = 0$

2.4:

For $w_{line} = (-1,6,0,0)$ we have:

- $l(w_{line}) = \frac{1}{3} \left(\sum_{i=1}^3 (6x_i - 1 + y_i)^2 \right) =$
 $\frac{1}{3} ((0 - 1 - 0)^2 + (6 - 1 - 3)^2 + (6 * 2 - 1 - 12)^2) =$
 $\frac{1}{3} (1 + 4 + 1) = 2$

2.5:

For $l_\lambda(w)$, $w \in \{w_{2.1}, w_{2.2}, w_{line}\}$ we have:

- $l_\lambda(w_{2.1}) = 0 + \lambda(1^2 + 2^2) = 5\lambda$
- $l_\lambda(w_{2.2}) = 0 + \lambda(3^2) = 9\lambda$
- $l_\lambda(w_{line}) = 0 + \lambda(6^2 + (-1)^2) = 37\lambda$

2.6:

Since the regularized loss of the discussed cubic polynomial is strictly less than the regularized loss of the cubic polynomial for all $\lambda > 0$, Therefore while minimizing the term $l_\lambda(w)$, we would prefer w_{cubic} over $w_{quadratic}$. Leading to the choice of the cubic polynomial over the quadratic one.

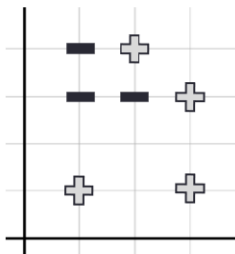
2.7:

For L1 regularization, both polynomials have regularized loss of 3λ , meaning this method prefers both polynomials equally. Since l1 regularization tends to zero out more coefficients, the quadratic polynomial is favored.

Part C:

Question 3

Initial figure:

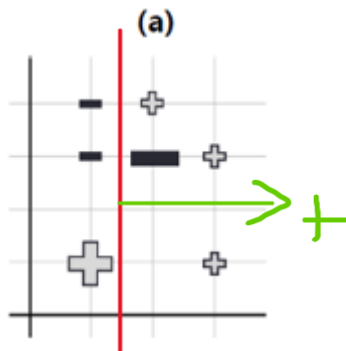


All figures:

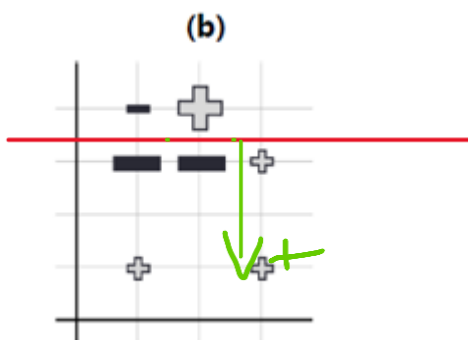


Possible figures:

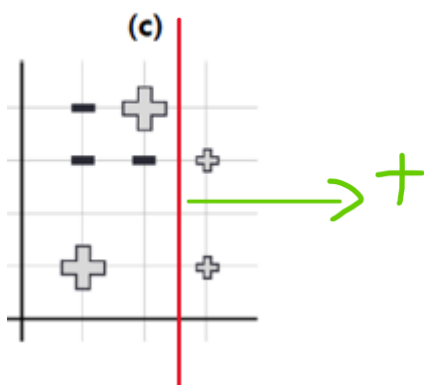
(a) For this split:



(b) For this split:



(c) For this split:



(d) + (e) No split can produce these distributions.

Question 4

4. We have informally argued that the AdaBoost algorithm uses the weighting mechanism to “force” the weak learner to focus on the problematic examples in the next iteration. In this question we will find some rigorous justification for this argument.

Show that the error of h_t w.r.t the distribution D^{t+1} is exactly $1/2$. That is, show that $\forall t \in [T]$

$$\sum_{i=1}^m D_i^{t+1} \mathbb{I}_{[y_i \neq h_t(x_i)]} = 1/2.$$

Proof:

Denote $R = \{x_i \mid h_t(x_i) \neq y_i\}$, consider: $\sum_{i=1}^m D_i^{t+1} \mathbb{I}_{[h_t(x_i) \neq y_i]} = ***$.

$$*** = \sum_{i \in R} D_i^{t+1} = \sum_{i \in R} \frac{D_i^t e^{-\alpha_t y_i h_t(x_i)}}{\sum_{j=1}^m D_j^t e^{-\alpha_t y_j h_t(x_j)}}.$$

Note: For all $x_i \in R$, we have that $y_i h_t(x_i) = -1$, therefore:

$$*** = \sum_{i \in R} \frac{D_i^t e^{\alpha_t}}{\sum_{j=1}^m D_j^t e^{-\alpha_t y_j h_t(x_j)}}.$$

$$\text{Consider: } e^{\alpha_t} = e^{\frac{1}{2} \ln\left(\frac{1}{\epsilon_t} - 1\right)} = \sqrt{\frac{1}{\epsilon_t} - 1} = **$$

$$\text{Consider: } \sum_{j=1}^m D_j^t e^{-\alpha_t y_j h_t(x_j)} = \sum_{i \in R} D_i^t e^{\alpha_t} + \sum_{i \notin R} D_i^t e^{-\alpha_t}$$

$$= \sum_{i \in R} D_i^t \sqrt{\frac{1}{\epsilon_t} - 1} + \sum_{i \notin R} D_i^t \sqrt{\frac{\epsilon_t}{\epsilon_t - 1}} = *$$

$$\text{Observe now: } *** = \sum_{i \in R} \frac{D_i^t \sqrt{\frac{1}{\epsilon_t} - 1}}{\sum_{i \in R} D_i^t \sqrt{\frac{1}{\epsilon_t} - 1} + \sum_{i \notin R} D_i^t \sqrt{\frac{\epsilon_t}{\epsilon_t - 1}}}.$$

Multiplying both the numerator the denominator by $\sqrt{\epsilon_t}$ we get:

$$*** = \sum_{i \in R} \frac{D_i^t \sqrt{1 - \epsilon_t}}{\sum_{i \in R} D_i^t \sqrt{1 - \epsilon_t} + \sum_{i \notin R} D_i^t \sqrt{\frac{\epsilon_t^2}{\epsilon_t - 1}}}$$

Notice that $\epsilon_t = \frac{\sum_{i \in R} D_i^t}{\sum_{i \in R} D_i^t + \sum_{i \notin R} D_i^t} \rightarrow \sum_{i \notin R} D_i^t = \frac{\sum_{i \in R} D_i^t}{\epsilon_t} - \sum_{i \in R} D_i^t$

Combining this with what we showed above we have:

$$*** = \sum_{i \in R} \frac{D_i^t \sqrt{1 - \epsilon_t}}{\sum_{i \in R} D_i^t \sqrt{1 - \epsilon_t} + \left(\frac{\sum_{i \in R} D_i^t}{\epsilon_t} - \sum_{i \in R} D_i^t \right) \sqrt{\frac{\epsilon_t^2}{\epsilon_t - 1}}}$$

Now dividing by $\sum_{i \in R} D_i^t$ we get:

$$*** = \frac{\sqrt{1 - \epsilon_t}}{\sqrt{1 - \epsilon_t} + \left(\frac{1}{\epsilon_t} - 1 \right) \sqrt{\frac{\epsilon_t^2}{\epsilon_t - 1}}}$$

Multiplying both numerator and denominator by $\sqrt{1 - \epsilon_t}$ we finally have:

$$*** = \frac{1 - \epsilon_t}{1 - \epsilon_t + \left(\frac{1}{\epsilon_t} - 1 \right) \epsilon_t} = \frac{1 - \epsilon_t}{1 - \epsilon_t + 1 - \epsilon_t} = \frac{1}{2}$$

Therefore proving what is needed.

Part D:

Question 5:

5. Recall the vanilla perceptron algorithm: For an input trainset $(x_1, y_1), \dots, (x_m, y_m)$

```
w = 0_d
while didn't separate trainset
  for i=1 to m
    y_i_hat = sign(w^T x_i)
    if y_i != y_i_hat
      w = w + eta y_i x_i
```

Prove that $\forall \eta > 0$ the perceptron algorithm will perform the same number of iterations, and will converge to a vector that points to the same direction.

Proof:

Let learning rates η_1, η_2 greater than zero, given an input training set $\{(x_i, y_i)\}_i^m$, we have for any iteration $i \in \mathbb{N}$:

$$w_i = \sum_{j=1}^i \eta_r y_j x_j \text{ for } r \in \{1, 2\}$$

Therefore, since the ratio between η_1 and η_2 is positive, w_i produced by either one will be in the same direction.

Regarding the number of iterations, consider for any iteration $n \in \mathbb{N}$:

$$y_{i_{pred}} = \text{sign}(w_n^T x_i) = \text{sign}\left(\sum_{j=1}^n \eta_r y_j x_j^T x_i\right) = \text{sign}\left(\sum_{j=1}^n y_j x_j^T x_i\right)$$

For any $r \in \{1, 2\}$, since $\eta_r > 0$.

Therefore predictions are invariant to any $\eta > 0$, at any iteration.

Meaning convergence is reached with the same number of iterations regardless of η .